Région Hauts-de-France

Cofinancé par l'Union européenne

realconseil
DIAGNOSTIC ACCOMPAGNEMENT FORMATION

SYMFONY 6 – MODIFICATION DU MOT DE PASSE

INTRODUCTION

Nous arrivons maintenant à modifier le nom et le pseudo d'un utilisateur, nous allons voir maintenant comment nous faisons pour modifier un mail et un mot de passe

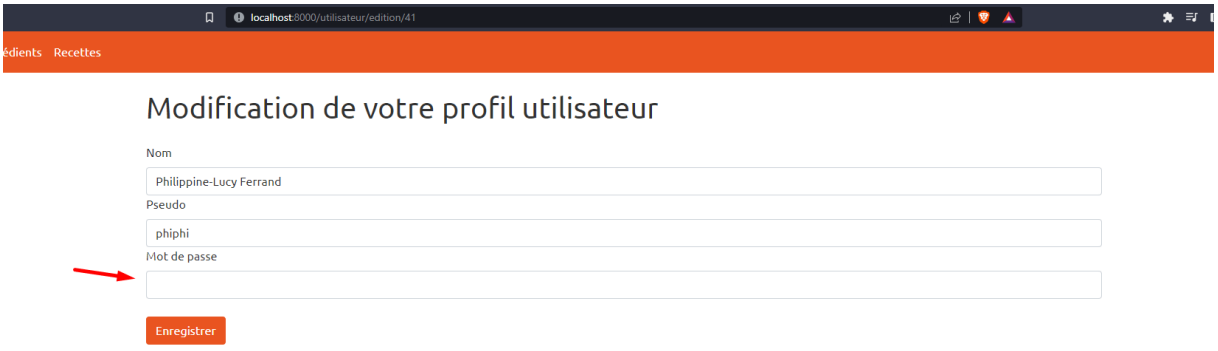
I) Modification du Usertype

Tout d'abord nous allons revenir sur notre modification de pseudo et mot de passe et nous allons demander à l'utilisateur de saisir son mot de passe pour valider sa modification

Retournons sur notre Usertype et collons le plainPassword que nous avons dans le registrationType et le modifier comme ceci :

```
48          ->add('plainPassword',PasswordType::class,[
49                  'attr'=>[
50                      'class'=>'form-control'
51                  ],
52                  'label'=>'Mot de passe',
53                  'label_attr'=>[
54                      'class'=>'form-label'
55                  ],
56          ])
```

Au niveau du visuel dans le navigateur nous avons le champ du mot de passe qui est revenu



Nous allons ensuite revenir sur notre UserController, il faudra utiliser le UserPasswordHasherInterface en injection de dépendances.

Une nouvelle fois n'oubliez pas vos use

```
#[Route('/utilisateur/edition/{id}', name: 'app_user_edit', methods:['GET','POST'])]
public function edit(UserRepository $userRepository,int $id,Request $request, EntityManagerInterface $manager, UserPasswordHasherInterface $hasher
{
    $user =$userRepository->findOneBy(["id"=>$id]);
```

Nous allons faire une condition imbriquée dans la condition déjà réalisé et grâce à notre injection de dépendance nous pourrons reprendre la méthode isPasswordValid() comme méthod pour vérifier si le mot de passe est valide de la façon suivante

```
43        $form->handleRequest($request);
44
45        if($form->isSubmitted() && $form->isValid()){
46
47            if($hasher->isPasswordValid($user, $form->getData()->getPlainPassword())){
48
49            }
50            $user = $form->getData();
51
52            $manager->persist($user);
53            $manager->flush();
54
55            $this->addFlash(
```

Nous allons faire ensuite notre logique en fonction de la condition « si le mot de passe est valide »

C'est-à-dire mettre l'ensemble de la logique qui a été faite à l'intérieur de celle-ci comme ci-dessous

```
41        $form = $this->createForm(UserType::class,$user);
42
43        $form->handleRequest($request);
44
45        if($form->isSubmitted() && $form->isValid()){
46
47            if($hasher->isPasswordValid($user, $form->getData()->getPlainPassword())){
48                $user = $form->getData();
49
50            $manager->persist($user);
51            $manager->flush();
52
53            $this->addFlash(
54                'success',
55                'Les informations de votre compte ont bien été modifiées'
56            );
57            return $this->redirectToRoute('app_recipe');
58            }
59
60        }
61
62        return $this->render('pages/user/edit.html.twig', [
63            'form'=>$form->createView(),
64        ]);
65    }
66 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS

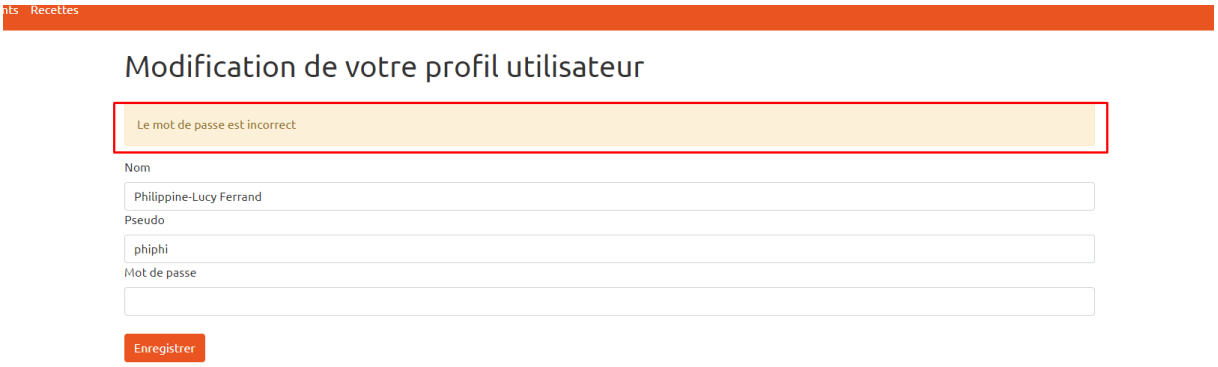
N'oublions pas d'avertir par message flash si l'utilisateur dans le cas où le message flash n'est pas le bon

```
52
53            $this->addFlash(
54                'success',
55                'Les informations de votre compte ont bien été modifiées'
56            );
57            return $this->redirectToRoute('app_recipe');
58            }
59            else{
60                $this->addFlash(
61                    'warning',
62                    'Le mot de passe est incorrect'
63                );
64            }
65        }
```

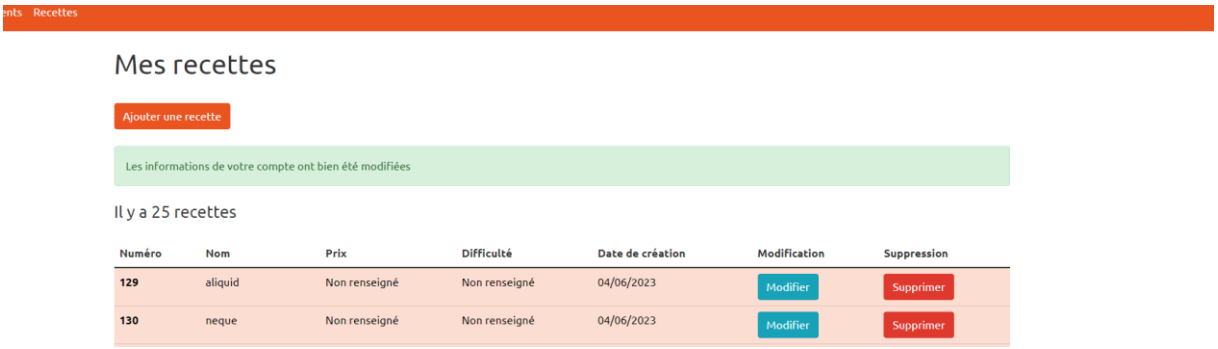
Faisons l'affichage du message flash warning dans le formulaire edit.html.twig du dossier User afin de faire afficher le message quand le mot de passe est incorrect

```
templates > pages > user > edit.html.twig > div.container > div.mt-4 > div.alert.alert-warning
1   {% extends 'base.html.twig' %}
2
3   {% block title %}Modification de l'utilisateur{% endblock %}
4
5   {% block body %}
6   <div class="container">
7       <h1 class="mt-4">Modification de votre profil utilisateur</h1>
8       <div class="mt-4">
9           {% for message in app.flashes('warning') %}
10          <div class="alert alert-warning mt-4">
11              {{ message }}
12          </div>
13          {% endfor %}
14          {{form(form)}}
15      </div>
16  </div>
17
18  {% endblock %}
19
```

Au niveau du visuel ça fonctionne bien et ça n'envoie rien en bdd au cas où le mot de passe n'est pas le bon



Quand le mot de passe est le bon nous avons ce rendu dans le navigateur



Et en BDD nous avons :

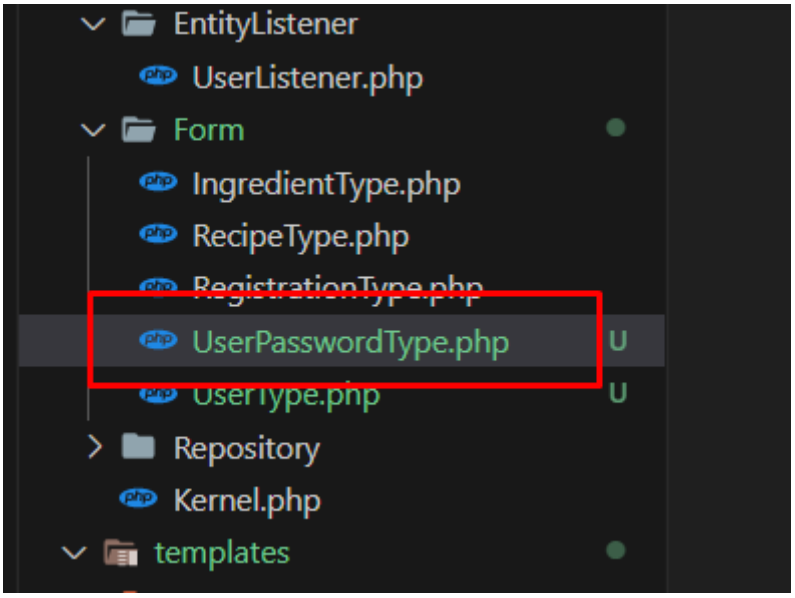
	id	email	roles	password	name	pseudo	created_at
Éditer Copier Supprimer	41	remy.guillaume@rousset.com	["ROLE_USER"]	$2y$13$fnRqU9ZczMpEsGENp7.FCevNMRGcrYE9kWMbQSUqwou...	Philippine-Lucy Ferrand	phiphine	2023-06-04 19:11:45
Éditer Copier Supprimer	42	rpons@leroux.com	["ROLE_USER"]	$2y$13$c9J5bpC/I5EfYm/VMQlku4AoNZWBFtVRV6/1e/GiRx...	Georges Le Guillaume	NULL	2023-06-04 19:11:45
Éditer Copier Supprimer	43	smillet@dbmail.com	["ROLE_USER"]	$2y$13$/5HZnCueddd0BI1tb1c5Uu0CsLiRmzdXaKXeHCl9nzC	Martin-Léon Nicolas	NULL	2023-06-04 19:11:46

Le changement a bien été fait de phiphi je suis passé à phiphine.

II) Le formulaire de la modification du mot de passe

Nous allons tout d'abord non pas avec la commande pour faire un formulaire mais plutôt à la main créer un UserPasswordType pour qu'il ne dépende pas d'une Entity

Donc créons ce fichier dans le dossier Form puisque c'est un formulaire



Nous allons faire donc notre formulaire dans ce fichier php :

- Balise php
- Namespace
- Use
- Class
- buildForm()

```php
<?php

namespace App\Form;


use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;


class UserType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder

            ->add('');
    }


}
```

De base nous avons donc ceci à créer à la main

Maintenant à notre formulaire nous allons add :

- plainPassword
- newPassword
- le submit

```php
->add('plainPassword', RepeatedType::class, [
    'type' => PasswordType::class,
    'first_options' => [
        'attr' => [
            'class' => 'form-control'
        ],
        'label' => 'Mot de passe',
        'label_attr' => [
            'class' => 'form-label  mt-4'
        ]
    ],
    'second_options' => [
        'attr' => [
            'class' => 'form-control'
        ],
        'label' => 'Confirmation du mot de passe',
        'label_attr' => [
            'class' => 'form-label  mt-4'
        ]
    ],
    'invalid_message' => 'Les mots de passe ne correspondent pas.'
])
```

```
            ->add('newPassword', PasswordType::class, [
                'attr' => ['class' => 'form-control'],
                'label' => 'Nouveau mot de passe',
                'label_attr' => ['class' => 'form-label mt-4'],
                'constraints' => [new Assert\NotBlank()]
            ])
            ->add('submit', SubmitType::class, [
                'attr' => [
                    'class' => 'btn btn-primary mt-4'
                ],
                'label' => 'Changer mon mot de passe'
            ]);
        }
    }
```

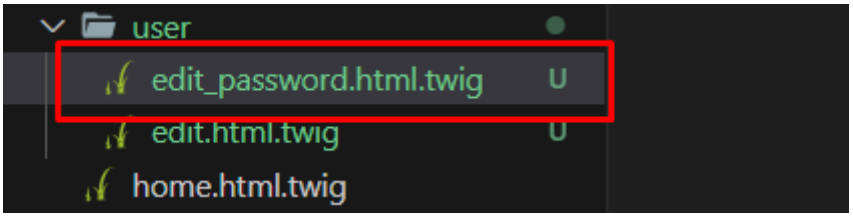
Maintenant que notre UserPasswordType est créé nous allons passer à la création d'une public funtion dans notre controller UserController

Pour le moment préparons juste la méthod pour qu'elle nous renvoi la view et rien d'autres, attention à ne pas oublier le décorateur pour la route

```
    #[Route('/utilisateur/edition-mot-de-passe/{id}',name: 'user_edit_password', methods:['GET','POST'])]
    public function editPassword() : Response
    {
        return $this->render('pages/user/edit_Password.html.twig');
    }
```

Nous appelons la view edit_Passwor.html.twig mais elle n'est pas créée dans le dossier pages

Allons y pour la créer comme nous savons le faire dans le dossier User qui se trouve dans le dossier Pages :



Dans ce fichier appelons le formulaire mais faisons bien les choses dans l'ordre

- Le extends
- Le block title
- Le h1
- Le message flash
- Le formulaire

templates > pages > user > edit_password.html.twig > ...

```
{% extends "base.html.twig" %}

{% block title %}Modification du mot de passe{% endblock title %}

{% block body %}
<div class="container">
    <h1 class="mt-4">Modification de votre profil utilisateur</h1>
    <div class="mt-4">
        {% for message in app.flashes('warning') %}
        <div class="alert alert-warning mt-4">
                {{ message }}
        </div>
        {% endfor %}
        {{form(form)}}
    </div>
</div>
{% endblock body %}
```

Faisons de suite la logique du formulaire dans notre controller dans la public function que nous avons créé juste avant

- On reprend nos injections de dépendance de la précédente function
- On voudra récupérer l'utilisateur en question (on fera notre query)
- Faire le createForm en liant notre function au formulaire
- Faire le createView au niveau du render pour passer la logique du formulaire à la view

```
#[Route('/utilisateur/edition-mot-de-passe/{id}',name: 'user_edit_password', methods:['GET','POST'])]
public function editPassword(UserRepository $userRepository,int $id,Request $request, EntityManagerInterface $manager, UserPasswordHasherInterface $hasher) : Response
{
    //récupération du user par son $id
    $user =$userRepository->findOneBy(["id"=>$id]);

    $form = $this->createForm(UserPasswordType::class,$user);

    return $this->render('pages/user/edit_password.html.twig',[
        'form'=>$form->createView()
    ]);
}
```

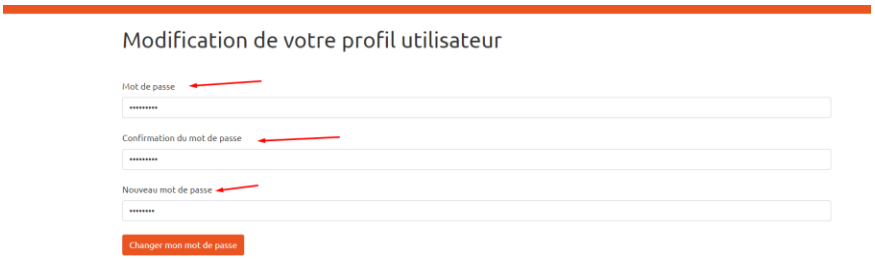
Au niveau du visuel notre formulaire s'affiche bien en fonction de la route

Nous allons donc continuer notre logique dans notre function dans le controller

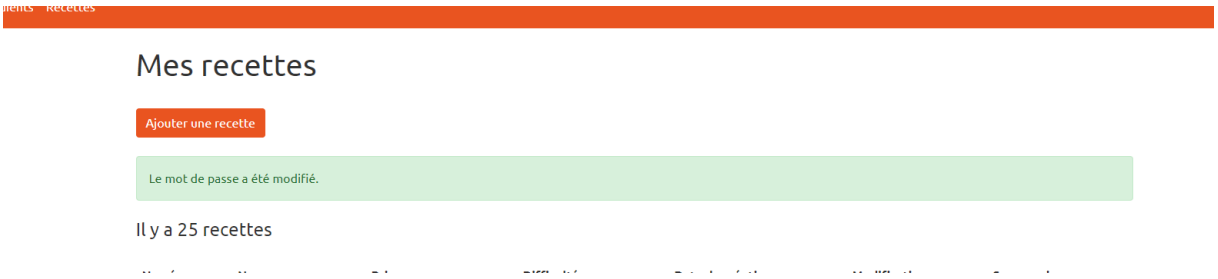
- Faire le handleRequest
- Faire la vérif si le formulaire est soumis et valide
- Faire à l'intérieur de la vérif on fait comme dans la function précédente le hashage du mot de passe
- Si c'est ok on dit que l'on modifie le plainPassword par le newPassword avec un message flash pour dire que le mot de passe a été modifié
- On fait un redirectToRoute vers la liste des recettes
- Else (sinon)
- On fait un message flash warning en disant que le mot de passe renseigné est incorect

```php
public function editPassword(UserRepository $userRepository,int $id,Request $request, EntityManagerInterface $manage
{
    //récupération du user par son $id
    $user =$userRepository->findOneBy(["id"=>$id]);

    $form = $this->createForm(UserPasswordType::class);

    $form->handleRequest($request);
    if($form->isSubmitted() && $form->isValid()){
        if($hasher->isPasswordValid($user,$form->getData()['plainPassword'])){

            $user->setPlainPassword(
                $form->getData()['newPassword']
            );

            $this->addFlash(
                'success',
                'Le mot de passe a été modifié.'
            );
            return $this->redirectToRoute('app_recipe');
        }else{
            $this->addFlash(
                'warning',
                "Le mot de passe renseigné est incorrect"
            );
        }
    }

    return $this->render('pages/user/edit_password.html.twig',[
        'form'=>$form->createView()
    ]);
```

Pour le visuel cela doit se passer de la façon suivante allons changer le mot de passe du user sur lequel nous sommes connecté

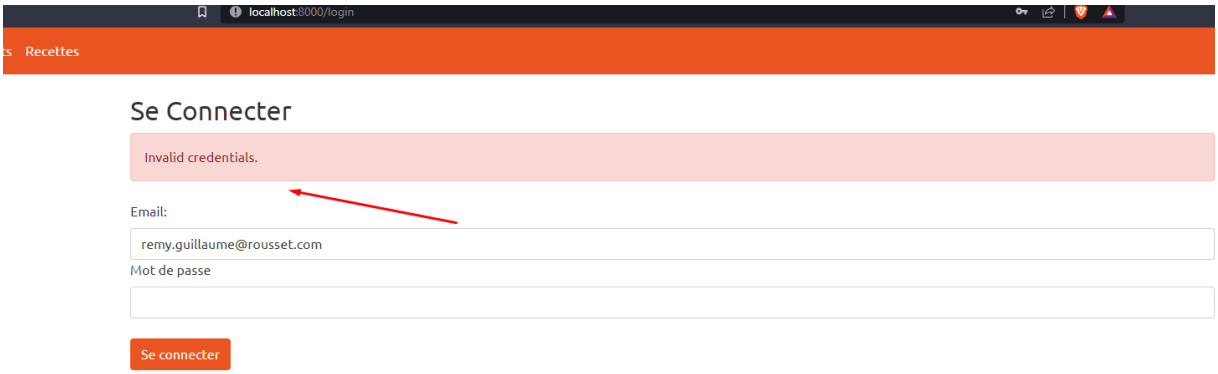


Cliquons sur le bouton changer mon mot de passe



On a bien ce retour

Maintenant nous allons nous déconnecter de ce compte et aller sur le formulaire du login pour se connecter avec le nouveau mot de passe



Nous ne sommes pas parvenu à nous connecter et un message d'erreur sous forme d'alerte nous revient

Effectivement nous n'avons pas envoyé la modification en bdd. Notre logique est presque bonne mais pas finie retournons donc dans notre controller et finissons la function

```
79        $form = $this->createForm(UserPasswordType::class);
80
81        $form->handleRequest($request);
82        if($form->isSubmitted() && $form->isValid()){
83            if($hasher->isPasswordValid($user,$form->getData()['plainPassword'])){
84
85                $user->setPassword(
86                    $hasher->hashPassword(
87                        $user,
88                        $form->getData()['newPassword']
89                    )
90                    );
91
92                $manager->persist($user);
93                $manager->flush();
94
95                $this->addFlash(
```

Changeons à nouveau le mot de passe et essayons après être déconnecté de se reconnecter avec le nouveau mot de passe. Cela fonctionne très bien